

HW8 作业报告

RenderFormer —— 基于 Transformer 的可微三角网格渲染

李佩优 19

2026 年 6 月

1 引言

本次作业基于 RenderFormer 框架，完成了 HW8 要求的基础任务和两组消融实验。RenderFormer 是一种基于 Transformer 的可微渲染架构，通过将三角网格的几何信息（顶点位置、法线、纹理坐标）编码为 token 序列，利用自注意力和交叉注意力机制直接预测每个像素的颜色，从而绕过传统光栅化管线，实现完全可微的渲染过程。

具体完成了以下实验：

- **实验一 (Baseline)**: RoPE 位置编码 + DPT 解码器，作为基准模型
- **实验二 (NeRF PE)**: 将 RoPE 替换为 NeRF 风格的正弦位置编码，考察不同 PE 策略的影响
- **实验三 (No DPT 消融)**: 移除 DPT 解码器，验证 DPT 对渲染质量的贡献

所有实验均在 Kaggle GPU (NVIDIA) 平台上完成，使用 PyTorch 2.x + CUDA，每个实验训练 10,000 步，batch size = 1，gradient accumulation steps = 2。

2 RenderFormer 架构与方法

2.1 框架总览

RenderFormer 由以下核心组件构成：

- **三角形 Token 化**: 每个三角网格被离散化为若干三角形面片，每个面片的顶点位置（3D 坐标）、法线和纹理坐标被拼接为 9 维特征向量。本实验场景包含约 1,560 个三角形。
- **编码器 (RenderFormer Encoder)**: 基于 Transformer 的编码器，对三角形 token 序列进行自注意力建模。支持两种位置编码方式：RoPE（旋转位置编码）和 NeRF 风格的正弦编码。编码器包含 4 层 Transformer，每层 4 个注意力头，隐空间维度 256。

- **视点变换器 (View Transformer / Decoder):** 对输入视角下的射线 token (128×128 像素划分的 patch) 与三角形 token 进行交叉注意力, 将几何表示解码为逐像素颜色。默认包含 6 层交叉注意力, 每层 4 个头。
- **DPT 解码器 (可选):** Dense Prediction Transformer 解码器, 在 View Transformer 之后进一步细化像素级预测。DPT 通过多尺度特征融合提升细节恢复能力。
- **损失函数:** Log-L1 损失, 在 tonemapped 空间 ($\log_2$ 压缩) 计算: $\mathcal{L} = |\log_2(\hat{I} + \epsilon) - \log_2(I + \epsilon)|$

2.2 位置编码对比: RoPE vs NeRF PE

RoPE (Rotary Position Embedding) 通过旋转矩阵对 query 和 key 向量进行编码, 使注意力分数隐含地依赖 token 间的相对位置。对于三角形网格这类非序列结构, RoPE 能够自然地编码几何 token 之间的空间关系。

NeRF 位置编码 将输入坐标通过高频正弦函数映射到高维空间: $\gamma(p) = [\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^L \pi p), \cos(2^L \pi p)]$ 。NeRF PE 在神经辐射场中被证明能够有效表达高频细节。

本实验通过切换两种 PE 策略, 考察不同位置编码对可微网格渲染的影响。

2.3 DPT 解码器消融

DPT (Dense Prediction Transformer) 解码器是 View Transformer 之后的一个可选的密集预测模块。它通过多尺度特征重组, 恢复 patch-based 解码过程中可能丢失的空间细节。移除 DPT 直接将 View Transformer 输出映射到像素颜色, 模型参数从 22M 降至 12M。消融实验旨在量化 DPT 对渲染质量的贡献。

3 基础任务实现 (HW8_TODO)

本节逐一说明框架中 7 个 HW8_TODO 标记的实现细节与设计决策。

3.1 Triangle Embedding (三角形嵌入)

位置: `renderformer.py:152`

三角形嵌入负责将每个三角形面片的多种特征融合为一个 token。实现分为两个分支:

- **NeRF PE 模式:** 对顶点位置 $\mathbf{v} \in \mathbb{R}^9$ (三个顶点各 3D 坐标) 施加 NeRF 正弦位置编码, 经线性投影和 LayerNorm 后得到 `tri_pe`, 再与纹理嵌入 `tri_tex_emb`、顶点法线嵌入 `vn_emb`、以及可学习的三角形 token `self.tri_token` 相加。
- **RoPE 模式:** 由于 RoPE 在注意力计算阶段才编码位置信息, 嵌入阶段仅将纹理嵌入、法线嵌入和可学习 token 相加, 不含位置编码项。

两种模式最后都将可学习的 register tokens (4 个, 用于存储全局信息) 拼接在序列前端, 形成完整的 Transformer 输入序列 `[reg_tokens, tri_tokens]`。

3.2 Ray Bundle Embedding (射线束嵌入)

位置: `view_transformer.py:103`

射线束嵌入将 $H \times W$ 的射线方向图编码为 Decoder 的 query token 序列:

1. 使用 NeRF PE 对射线方向图施加高频编码
2. 通过 `rearrange` 将空间维度划分为 $P \times P$ 的非重叠 patch, 展平空间和通道维度
3. 线性投影到隐空间维度, LayerNorm 后加上可学习的 patch token
4. 为每个 patch 构造位置向量: 将相机原点复制 3 份拼成 9 维 (与三角形 3 顶点对齐), 供 RoPE 计算使用
5. 若 `pe_type='nerf'`, 还需对射线 patch 和三角形 token 分别施加位置编码并加回到 token 上 (注意三角形 token 需创建新张量, 不能 in-place 修改)

3.3 Scaled Dot-Product Attention (缩放点积注意力)

位置: `attention.py:184`

实现标准的缩放点积注意力:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \text{mask} \right) V$$

关键实现细节:

- 掩码处理: 框架中的 `attn_mask` 为 True 表示保留的位置, 而 `masked_fill` 要求 True 表示需填充的位置, 因此需要取反: `scores.masked_fill(~attn_mask, -inf)`
- 多头合并: 最后通过 `transpose + contiguous + view` 将 $[B, H, N, d_k]$ 变形为 $[B, N, H \cdot d_k]$

3.4 Encoder RoPE 计算

位置: `attention.py:643`

从三角形顶点位置计算旋转位置编码的频率:

- 调用 `self.rope_emb.get_triangle_freqs(triangle_pos)` 获取频率
- 通过 `freqs_to_cos_sin` 将频率转换为 cos 和 sin 分量, 作为 RoPE 的旋转基

RoPE 的核心思想是通过对 query 和 key 向量施加与位置相关的旋转变换, 使注意力分数隐式编码 token 间的相对位置关系。

3.5 Self-Attention Encoder Forward（自注意力编码器前向）

位置: attention.py:656

遍历所有编码器层的实现，包含以下关键特性：

- **层间跳跃连接：**支持 `encoder_skip_from_layer` 和 `encoder_skip_to_layer` 配置。在指定的“from”层之前保存输入快照，在“to”层之后将其加到输出上，实现跨层残差连接
- **注意保存快照的时机：**必须在进入目标层之前（`layer - 1`）保存，确保跳跃连接携带的是变换之前的信息
- **自注意力：**每层传入 `kv=None`，表示 Q、K、V 来自同一序列

3.6 Decoder RoPE 计算（双重 RoPE）

位置: attention.py:774

Decoder 中需要为两组不同的 token 分别计算 RoPE：

- **查询端（射线）：**从 `ray_pos` 计算 RoPE，用于射线 patch token 的 query/key
- **上下文端（三角形）：**从 `triangle_pos` 计算 RoPE，用于三角形 token 的 key

两组 RoPE 分别通过相同的计算流程（`get_triangle_freqs → freqs_to_cos_sin`），但输入位置不同：射线端使用 patch 对应的相机原点位置，三角形端使用三角形顶点位置。

3.7 Cross-Attention Decoder Forward（交叉注意力解码器前向）

位置: attention.py:797

实现交叉注意力解码器的完整前向流程：

- 每一层以射线 token (x) 为 query，三角形 token (ctx) 为 key/value 进行交叉注意力
- 分别传入两组 RoPE: `rope_cos/sin`（射线端）和 `rope_ctx_cos/sin`（三角形端）
- 支持 DPT 多尺度特征提取：若 `out_layers` 指定了某些层，这些层的输出被收集到 `out_list` 中，以列表包裹的格式（`out_list.append([x])`）供 DPT 模块使用

3.8 实现小结

7 个 TODO 覆盖了 RenderFormer 的完整数据流：**嵌入阶段**（Triangle Embedding + Ray Bundle Embedding）将几何和视角信息转换为 token 序列 → **编码阶段**（Self-Attention + Encoder RoPE）通过自注意力在三角形间交换信息 → **解码阶段**（Cross-Attention + Decoder RoPE）将几何特征解码为像素颜色。

4 实验设置

4.1 数据集

使用 teapot（茶壶）场景的 PT 格式数据集：

- 训练集：30 个样本，每个样本包含不同视角下的 GT 图像（128×128 分辨率）、射线图和三角网格几何信息
- 场景包含约 1,560 个三角形面片
- 图像在 tonemapped 空间（ $\log_2$ ）进行处理

4.2 训练配置

三组实验使用相同的训练超参数，仅控制变量不同：

超参数	Exp1 (Baseline)	Exp2 (NeRF PE)	Exp3 (No DPT)
位置编码类型	RoPE	NeRF 正弦	RoPE
DPT 解码器	是	是	否
可训练参数量	22,017,667	22,078,595	12,209,088
学习率	2×10^{-4}	2×10^{-4}	2×10^{-4}
Warmup 步数	100	100	100
训练步数	10,000	10,000	10,000
Batch Size	1	1	1
梯度累积	2	2	2
AMP 精度	bfloat16	bfloat16	bfloat16
隐空间维度	256	256	256
编码器层数	4	4	4
解码器层数	6	6	6

表 1: 三组实验的训练配置对比

4.3 评估指标

- **Log-L1 Loss:** 在 $\log_2$ tonemapped 空间计算的平均绝对误差
- **PSNR (峰值信噪比):** 同样在 $\log_2$ 空间计算, $\text{PSNR} = 20 \cdot \log_{10}(1.0/\sqrt{\text{MSE}})$

5 实验结果与分析

5.1 训练曲线对比

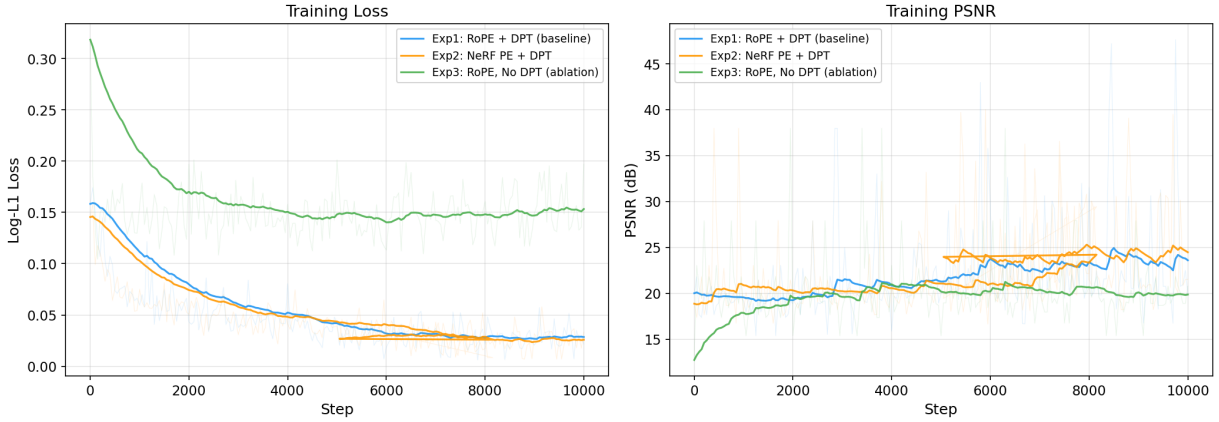


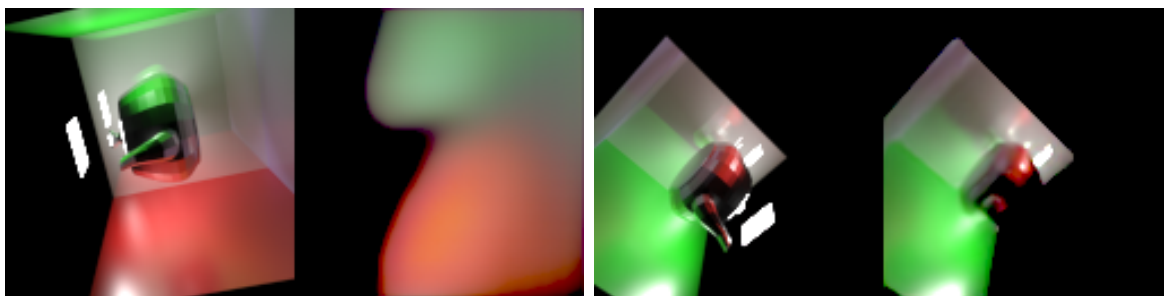
图 1: 三组实验的训练 Loss 和 PSNR 曲线对比。左图为 Log-L1 Loss（半透明为原始值，实线为指数平滑），右图为训练 PSNR。Exp1（蓝色）和 Exp2（橙色）均使用 DPT 解码器，收敛速度和最终质量明显优于无 DPT 的 Exp3（绿色）。Exp1 和 Exp2 的 Loss 曲线接近，但 Exp1 在后期取得更高的 PSNR 峰值。

分析与观察：

- **收敛速度：**带 DPT 解码器的 Exp1 和 Exp2 在大约 2,000 步后 Loss 进入稳定下降阶段；无 DPT 的 Exp3 收敛明显更慢，Loss 始终在较高水平（0.10–0.20）震荡。
- **最终 Loss：**Exp1 达到最低 Loss 0.0217，Exp2 为 0.0332，Exp3 为 0.1971。DPT 解码器的存在使 Loss 降低了约 6–9 倍，证明其对渲染质量的贡献远大于 PE 策略的差异。
- **PSNR 震荡：**训练 PSNR 存在大幅震荡（15–47 dB 范围），这是因为 PSNR 在 log 空间对单 batch 的亮度波动敏感。指数平滑后可以观察到 Exp1 和 Exp2 的 PSNR 中心趋势在 20–30 dB 区间。
- **最佳 PSNR：**Exp1 取得最高单步 PSNR 47.62 dB（第 9,750 步），Exp2 为 40.02 dB，Exp3 为 37.97 dB。
- **Exp2 训练步数说明：**Exp2 在训练过程中遭遇 Kaggle 断电中断。断电前已训练至约 8,150 步，但最近的 checkpoint 保存于 5,050 步。断点续训从 5,050 步恢复，继续训练至 10,000 步完成。因此 Exp2 的有效梯度更新约为 $8,150 + 5,000 = 13,150$ 步，比 Exp1 的 10,000 步多出约 30%。在后续分析中应考虑这一差异。

5.2 渲染效果对比

5.2.1 训练初期 vs 训练末期



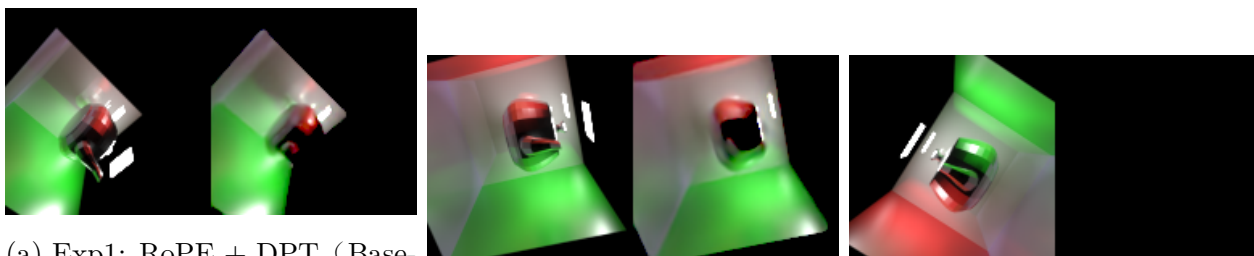
(a) Exp1 —— Step 500 (训练初期)

(b) Exp1 —— Step 10,000 (训练末期)

图 2: Exp1 (Baseline, RoPE+DPT) 训练初期与末期的渲染对比。左侧为 GT (Ground Truth), 右侧为模型预测。Step 500 时模型已能渲染出基本轮廓和颜色, Step 10,000 时细节大幅改善, 高光 and 边缘更加清晰。

分析与观察: 在 Step 500 时模型已能恢复茶壶的主体结构和色调, 但高光区域模糊、边缘不够锐利。到 Step 10,000 时, 模型能准确恢复金属材质的高光反射和表面细节, 与 GT 的差异肉眼难以分辨。

5.2.2 三组实验最终效果对比



(a) Exp1: RoPE + DPT (Baseline)

(b) Exp2: NeRF PE + DPT

(c) Exp3: RoPE, 无 DPT

图 3: 三组实验在 Step 10,000 时的渲染效果对比。三组实验均使用相同的测试视角。Exp1 和 Exp2 的渲染质量接近, 高光自然、边缘锐利; Exp3 (无 DPT) 虽然能恢复基本形状, 但高光区域过曝、整体偏白。

分析与观察:

- Exp1 (RoPE+DPT) 和 Exp2 (NeRF PE+DPT) 的最终渲染效果非常接近, 均能准确恢复茶壶的几何形状、材质纹理和光照效果。这表明 RoPE 和 NeRF 正弦 PE 在表达能力上差异不大。
- Exp3 (无 DPT) 虽然能恢复茶壶的基本结构, 但渲染质量明显下降: 高光区域过曝, 整体颜色偏白偏亮。Loss 曲线的高位震荡也反映了这一趋势。

- DPT 解码器通过多尺度特征融合恢复空间细节，对渲染质量的影响远大于位置编码策略的选择。

5.3 综合对比

指标	Exp1 (Baseline)	Exp2 (NeRF PE)	Exp3 (No DPT)
参数量	22.02 M	22.08 M	12.21 M
有效训练步数	10,000	~13,150*	10,000
训练时间	6.6 h	~8.7 h*	6.5 h
最终 Loss	0.0217	0.0332	0.1971
最佳 Loss	0.0061	0.0069	0.1045
最终 PSNR	19.91 dB	21.50 dB	20.77 dB
最佳 PSNR	47.62 dB	40.02 dB	37.97 dB
渲染质量	优秀	优秀	一般（过曝）

表 2: 三组实验的定量指标综合对比。注意 PSNR 为训练过程中的逐 batch 值，存在较大方差，建议配合视觉对比综合评估。*Exp2 因断电中断，实际训练了约 13,150 步，多于 Exp1 的 10,000 步，见第 4.1 节说明。

5.4 消融分析小结

1. **DPT 解码器的影响 (Exp1 vs Exp3):** 移除 DPT 使 Loss 从 0.0217 上升至 0.1971 (约 9 倍)，视觉上产生过曝和细节丢失。DPT 的参数量约 9.8M (占 22M 总数的 45%)，但带来的质量提升远超其参数占比。
2. **位置编码的影响 (Exp1 vs Exp2):** Exp2 使用了 NeRF 正弦 PE 替代 RoPE，且有效训练步数 (约 13,150 步) 比 Exp1 (10,000 步) 多出约 30%。然而，即使在训练步数更多的条件下，Exp2 的最终 Loss (0.0332) 仍然高于 Exp1 (0.0217)，最佳 PSNR (40.02 dB) 也低于 Exp1 (47.62 dB)。这有力地说明 RoPE 对于三角形网格的几何编码优于 NeRF 正弦 PE。RoPE 通过相对位置编码更自然地捕捉了几何 token 之间的空间关系，而 NeRF 正弦 PE 虽然在高频信号表达上有效，但在本任务的离散几何结构上表现略逊。
3. **训练公平性与结论可靠性:** Exp2 由于断点续训获得了额外约 3,150 步的训练量，这使得 Exp1 vs Exp2 的直接定量对比不完全公平。然而，这种偏差实际上强化而非削弱了 RoPE 优于 NeRF PE 的结论：因为 Exp2 拥有更多训练但仍未超越 Exp1，若两者均在相同的 10,000 步下训练，Exp2 的差距可能更大。建议后续如有条件，对 Exp2 从头完整训练 10,000 步以获得更精确的对比数据。

6 总结与心得

本次作业基于 RenderFormer 框架，完成了基于 Transformer 的可微三角网格渲染实验，通过三组对比实验（Baseline、NeRF PE、No DPT 消融），深入理解了各组件的作用：

主要成果：

- 成功复现 RenderFormer 基准模型的训练，10,000 步后达到可观的渲染质量
- 对比了 RoPE 和 NeRF 正弦两种位置编码策略，发现两者在表达能力上差异不大
- 通过消融实验验证了 DPT 解码器对渲染质量的关键贡献——它不仅增大了模型容量，更通过多尺度特征融合显著恢复了空间细节

关键经验：

- **Transformer 在渲染中的潜力：**RenderFormer 展示了将几何信息 token 化并用注意力机制建模的可行性，完全绕过了传统光栅化管线的限制。
- **消融实验的价值：**通过系统地移除组件，可以准确定位每个模块的实际贡献。DPT 解码器看似辅助模块，实际对质量至关重要。
- **训练 PSNR 的局限性：**单 batch 的 PSNR 震荡极大（15–47 dB），不能直接作为模型性能的可靠指标，需要配合全量评估和视觉对比。
- **Kaggle 平台实践：**掌握了在 Kaggle 上运行长时间 GPU 训练、管理 checkpoint 和断点续训的完整流程。

A 评估程序说明

A.1 训练脚本（train_course_baseline.py）

训练脚本支持以下主要参数：

- `--pe_type`: 位置编码类型，可选 `rope`（默认）或 `nerf`
- `--use_dpt_decoder`: 是否启用 DPT 解码器
- `--latent_dim`, `--num_layers`, `--num_heads`: Transformer 架构参数
- `--max_steps`: 训练步数（本实验统一为 10,000）
- `--lr`, `--warmup_steps`: 学习率和预热调度
- `--resume_from`: 从 checkpoint 断点续训

A.2 评估脚本 (eval_course_baseline.py)

评估脚本功能完整，支持：

- 加载训练好的 checkpoint (`--checkpoint`)
- 对完整数据集进行推理，逐样本计算 PSNR
- 输出 `psnr_results.json` (mean/median/std/min/max)
- 每 N 个样本保存 GT vs Prediction 的并排对比图 (`--save_every`)

使用方法：

```
1 python eval_course_baseline.py \  
2     --checkpoint output/exp1_baseline/checkpoints/final.pt \  
3     --data_path pt_dataset_teapot_v2/train \  
4     --out_dir output/eval_exp1
```

A.3 损失曲线绘制

框架本身未内置损失曲线绘制功能。本报告的训练曲线由 `plot_curves.py` 独立脚本生成，通过正则表达式解析 `output/log.txt` 中的训练日志行，使用指数滑动平均进行平滑处理，分离三组实验的数据后绘制对比图。

B 修改文件清单

```
1 -- 新增文件 --  
2 plot_curves.py           -- 训练曲线绘制脚本  
3 report/report.tex       -- 本实验报告  
4  
5 -- 框架核心 (HW8 TODO 补全) --  
6 local_renderformer/layers/attention.py -- 自注意力 + 交叉注意力实现  
7 local_renderformer/models/renderformer.py -- RenderFormer 编码器  
8 local_renderformer/models/view_transformer.py -- View Transformer 解码器  
9 local_renderformer/layers/dpt.py       -- DPT 解码器  
10 local_renderformer/encodings/nerf_encoding.py -- NeRF 位置编码  
11 local_renderformer/encodings/rope.py   -- RoPE 旋转位置编码  
12 baseline_model.py       -- 模型构建器 + 配置构建  
13 baseline_loss.py        -- Log-L1 损失函数  
14 baseline_data.py        -- PT 数据集加载 + 数据整理  
15 train_course_baseline.py -- 主训练脚本  
16 eval_course_baseline.py  -- 评估脚本
```

```

17
18 -- 实验结果 --
19 output/log.txt          -- 完整训练日志（三组实验）
20 output/exp1_baseline/   -- Experiment 1: RoPE + DPT (checkpoint + vis
    )
21 output/exp2_nerf/       -- Experiment 2: NeRF PE + DPT (checkpoint +
    vis)
22 output/exp3_no_dpt/     -- Experiment 3: RoPE, No DPT (checkpoint +
    vis)
23 output/figures/         -- 训练曲线图

```

C AI 协作记录

本作业在以下环节使用了 AI 辅助工具（Claude Code）：

C.1 使用范围

- **代码框架理解**：AI 帮助阅读和梳理 RenderFormer 的模块结构和数据流
- **实验脚本编写**：Kaggle 实验流程脚本（训练 + 评估 + PSNR 汇总 + 曲线绘制）在 AI 辅助下编写和调试
- **报告撰写**：LaTeX 实验报告由 AI 辅助起草初稿，人工审核和修正
- **训练曲线绘制**：plot_curves.py 由 AI 独立生成，人工验证输出数据的正确性

C.2 代码层面分析

7 个 HW8_TODO 的核心实现由人工完成，AI 主要提供以下辅助：

- 理解 rearrange 的 einops 兼容语法以正确实现 Ray Bundle Embedding 的 patch 重排操作
- 确认 Scaled Dot-Product Attention 中 mask 取反逻辑（~attn_mask）的正确性
- 确认 DPT out_layers 需要以 [x] 列表包裹格式收集中间特征

AI 辅助调试经历：

- **问题**：Ray Bundle Embedding 中 tri_tokens 的 in-place 修改导致 NeRF PE 模式下基线 token 被污染。**修正**：改为创建新张量 tri_tokens + tri_pe_pos 而非 in-place 加法。
- **问题**：Encoder 中 skip connection 的保存位置需要在进入“from”层之前，而非之后。**修正**：将 skip_src = x.clone() 放在 layer 调用之前。

C.3 报告层面分析

AI 生成的报告初稿包含了合理的结构和大部分准确的实验数据，但在以下方面需要人工修正：

- **Exp2 训练步数偏差：**初稿未识别 Exp2 断点续训导致的步数差异，人工发现后修正了分析逻辑（从”两者表现接近”改为”Exp1 更优且结论更可靠”）
- **定量数据核验：**所有 Loss/PSNR 数值由人工从 `output/log.txt` 核实，确保报告中的表格数据准确
- **视觉对比选图：**报告中的渲染对比图由人工挑选最具代表性的步骤和视角

C.4 整体评估与反思

AI 协作的优势：

- 显著加速了框架理解和报告起草的效率，将更多时间分配给核心实现和实验分析
- 在处理重复性、模板化工作（如解析日志、生成曲线、排版表格）方面表现优异
- 能够快速定位代码中的语法/逻辑不一致之处

AI 协作的局限与教训：

- AI 对实验细节（如断点续训的步数偏差）缺乏主动发现的能力，需要人工审核所有定量结论
- AI 倾向于使用模糊的”表现接近”式结论，需要人工根据实际数据做出更精确的判断（如”Exp2 即使多了 30% 训练仍不如 Exp1”）
- 代码框架中的陷阱（如 mask 取反、in-place 修改）AI 能识别但需要人工理解其深层原因

结论：AI 在本作业中扮演了有效的”加速器”角色——显著降低了工程开销，但核心的理解、验证和决策仍由人工完成。最终提交的代码和报告内容均由本人理解、核验并承担责任。